

Assignment #1: 虚拟机，Shell & 大语言模型

Updated 1745 GMT+8 Sep 8, 2025

2025 fall, Compiled by <窦兆文><元培学院>

作业的各项评分细则及对应的得分

标准	等级	得分
按时提交	完全按时提交：1分 提交有请假说明：0.5分 未提交：0分	1 分
源码、耗时（可选）、 解题思路（可选）	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于2个：0分	1 分
AC代码截图	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于：0分	1 分
清晰头像、PDF文件、 MD/DOC附件	包含清晰的Canvas头像、 PDF文件以及MD或DOC格式的附件：1分 缺少上述三项中的任意一项：0.5分 缺失两项或以上：0分	1 分
学习总结和个人收获	提交了学习总结和个人收获：1分 未提交学习总结或内容不详：0分	1 分
总得分： 5	总分满分：5分	

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge， Codeforces， LeetCode等平台上获得Accepted）。请将这些信息连同显示

“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **课程平台： **课程网站位于Canvas平台（<https://pku.instructure.com>）。该平台将在第2周选课结束后正式启用。在平台启用前，请先完成作业并将作业妥善保存。待Canvas平台激活后，再上传你的作业。
3. **提交安排： **提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
4. **延迟提交： ****如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E27653: Fraction类

<http://cs101.openjudge.cn/pctbook/E27653/>

请练习用OOP方式实现。

思路：

gcd(a, b) 函数：使用辗转相除法计算最大公约数，用于化简分数

Fraction 类：

init：构造函数，接收分子和分母，并自动化简

simplify：化简分数为最简形式（约去最大公约数，确保分母为正）

add：实现分数加法，使用通分公式： $a/b + c/d = (a \times d + c \times b) / (b \times d)$

str：返回"分子/分母"格式的字符串

代码：

```

def gcd(a, b):

    while b != 0:
        a, b = b, a % b
    return a


class Fraction:
    def __init__(self, numerator, denominator):

        if denominator == 0:
            raise ValueError
        self.numerator = numerator
        self.denominator = denominator

        self.simplify()

    def simplify(self):

        common_divisor = gcd(abs(self.numerator), abs(self.denominator))

        self.numerator //= common_divisor
        self.denominator //= common_divisor

        if self.denominator < 0:
            self.numerator = -self.numerator
            self.denominator = -self.denominator

    def __add__(self, other):

        new_numerator = self.numerator * other.denominator + other.numerator * self.denominator
        new_denominator = self.denominator * other.denominator

        return Fraction(new_numerator, new_denominator)

    def __str__(self):
        return f"{self.numerator}/{self.denominator}"


nums = list(map(int, input().split()))
n1, d1, n2, d2 = nums[0], nums[1], nums[2], nums[3]

frac1 = Fraction(n1, d1)

```

```
frac2 = Fraction(n2, d2)
```

```
result = frac1 + frac2
```

```
print(result)
```

代码运行截图

#50259092提交状态

[查看](#)[提交](#)[统计](#)[提问](#)

状态: **Accepted**

源代码

```
def gcd(a, b):  
    """计算最大公约数"""  
    while b != 0:  
        a, b = b, a % b  
    return a  
  
class Fraction:
```

基本信息

#: 50259092

题目: E27653

提交人: dzw

内存: 3656kB

时间: 22ms

语言: Python3

提交时间: 2025-10-08 14:54:08

M1760.袋子里最少数目的球

binary search, <https://leetcode.cn/problems/minimum-limit-of-balls-in-a-bag/>

思路:

二分查找目标: 在 $[1, \max(\text{nums})]$ 范围内二分查找最小的最大开销

判断函数: 对于给定的目标开销 target , 计算需要多少次操作才能让所有袋子的球数 $\leq \text{target}$

操作次数计算:

如果一个袋子有 n 个球, 要分成每袋最多 target 个球

需要的操作次数 $= \lceil n / \text{target} \rceil - 1 = (n - 1) // \text{target}$

例如: 9个球分成最多3个一袋, 需要 $\lceil 9/3 \rceil = 3$ 个袋子, 即 $3-1 = 2$ 次操作

代码:

```

class Solution:
    def minimumSize(self, nums, maxOperations):
        def canAchieve(target):
            operations = 0
            for num in nums:
                if num > target:
                    operations += (num - 1) // target
                    if operations > maxOperations:
                        return False
            return operations <= maxOperations

        left, right = 1, max(nums)
        result = right

        while left <= right:
            mid = (left + right) // 2
            if canAchieve(mid):
                result = mid
                right = mid - 1
            else:
                left = mid + 1

        return result

```

代码运行截图

[题目描述](#) | [通过](#) × | [题解](#) | [提交记录](#)

[← 全部提交记录](#)

通过 61 / 61 个通过的测试用例

[官方题解](#)
[写题解](#)

 Youthful Spencezwr 提交于 2025.10.08 15:11



面向在校生的优惠升级方案
[→](#)

完成认证享 1 元/天升级 Plus 会员，享更多学业及职业成长帮助

⌚ 执行用时分布
 [ⓘ](#)

851 ms | 击败 **28.13%**

[🌟 复杂度分析](#)

💻 消耗内存分布
 [ⓘ](#)

19.42 MB | 击败 **46.88%**

M04135: 月度开销

binary search, <http://cs101.openjudge.cn/pctbook/M04135/>

思路:

问题转化: 将N天分成M个连续段, 使得最大段和最小

二分查找: 答案范围是 $[\max(\text{开销}), \text{sum}(\text{开销})]$

下界: 至少要能容纳最大的单日开销

上界: 所有开销放在一个月

贪心验证: 给定预算上限, 判断能否分成 $\leq M$ 个月

从前往后贪心分配, 当前月累积开销不超过上限

如果超过就开始新的一个月

代码:

```

def can_divide(expenses, m, max_budget):
    months = 1
    current_sum = 0

    for expense in expenses:
        if current_sum + expense <= max_budget:
            current_sum += expense
        else:
            months += 1
            current_sum = expense

            if months > m:
                return False

    return True

```

```

def minimize_max_expense(n, m, expenses):
    left = max(expenses)
    right = sum(expenses)

    result = right

    while left <= right:
        mid = (left + right) // 2

        if can_divide(expenses, m, mid):
            result = mid
            right = mid - 1
        else:
            left = mid + 1

    return result

```

```

n, m = map(int, input().split())
expenses = []
for _ in range(n):
    expenses.append(int(input()))

print(minimize_max_expense(n, m, expenses))

```

代码运行截图

 **CS101 / 计算思维算法实践**

[题目](#) [排名](#) [状态](#) [提问](#)

#50259671提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
def can_divide(expenses, m, max_budget):  
    """  
    判断能否在max_budget的预算限制下, 将expenses分成不超过m个月  
  
    贪心策略: 从前往后遍历, 尽可能让每个月包含更多天数 (但不超过预算)  
    """  
    months = 1 # 至少需要1个月  
    current_sum = 0 # 当前月的累加和
```

基本信息

#: 50259671
题目: M04135
提交人: 袁兆文
内存: 7556kB
时间: 380ms
语言: Python3
提交时间: 2025-10-08 15:16:24

M27300: 模型整理

sortings, AI, <http://cs101.openjudge.cn/pctbook/M27300/>

思路:

parse_parameter(param_str):

将参数量字符串转换为统一的数值 (以M为单位)

'110M' → 110

'1.3B' → 1300

用于排序比较

使用字典存储模型, key是模型名称, value是参数量列表

对模型名称进行字典序排序

对每个模型的参数量按数值大小排序

代码:


```

def parse_parameter(param_str):
    if param_str.endswith('M'):
        return float(param_str[:-1])
    elif param_str.endswith('B'):
        return float(param_str[:-1]) * 1000
    return 0

def main():
    n = int(input())
    models = {}

    for _ in range(n):
        model_info = input().strip()
        parts = model_info.split('-')
        model_name = parts[0]
        param = parts[1]

        if model_name not in models:
            models[model_name] = []
        models[model_name].append(param)

    sorted_model_names = sorted(models.keys())

    for model_name in sorted_model_names:
        params = models[model_name]
        params.sort(key=parse_parameter)
        params_str = ', '.join(params)
        print(f"{model_name}: {params_str}")

if __name__ == "__main__":
    main()

```

代码运行截图

状态: Accepted

源代码

```
def parse_parameter(param_str):  
    """  
    解析参数量字符串, 返回数值 (以M为单位)  
    例如: '110M' -> 110, '1.3B' -> 1300  
    """  
    if param_str.endswith('M'):  
        return float(param_str[:-1])  
    elif param_str.endswith('B'):
```

基本信息

#: 50259798

题目: M27300

提交人: 窦兆文

内存: 3688kB

时间: 26ms

语言: Python3

提交时间: 2025-10-08 15:21:49

Q5. 熟悉云虚拟机Linux环境与大语言模型（LLM）本地部署

本项目包括两个任务：

- 1) 通过云虚拟机（如 <https://clab.pku.edu.cn/> 提供的资源）熟悉Linux系统操作环境。
- 2) 完成大语言模型（LLM）的本地部署与功能测试。

LLM 部署可选择使用图形化工具（如 LM Studio, <https://lmstudio.ai>）以简化配置流程，提升部署效率。部署完成后，需对模型进行实际能力测试。

测试内容包括：从主流在线编程评测平台（如 OpenJudge、Codeforces、LeetCode 或洛谷等）选取若干编程题目，提交由本地部署的 LLM 生成的代码解决方案，并确保其能够通过全部测试用例，获得“Accepted”状态。选题时应避免与已知可被 AI 正确解答的题目重复。当前已确认可通过的 AI 解题列表可参考以下 GitHub 仓库：

https://github.com/GMyhf/2025spring-cs201/blob/main/AI_accepted_locally.md

请提供你的项目进展，内容应该包括：关键操作步骤的截图以及遇到的技术问题及相应的解决方法。这将有助于全面掌握项目推进情况，并为后续优化与扩展提供依据。

我利用ollma在配置为i7-12700h和3070laptop的笔记本上，本地部署了参数量为8b的deepseek-r1模型，并通过chrome浏览器的Page Assist插件为其提供图形化界面，选取### M27300: 模型整理 sortings, AI, <http://cs101.openjudge.cn/pctbook/M27300/>，测试其能力，得到如下代码

```
{
  "name": "M2",
  "content": [
    {
      id: 'prompt1',
      role: string,
      content: str = input().strip() or something else.
    }
  ]
}
```

But note that in this case we are not see through but rather a specific question. I'm sorry, I c

并在此之后陷入中文与英文不断交替的混乱语言中，完全无法给出有价值的回答，可见参数量较小的模型在有限的硬件条件下，无法解答较复杂的问题

Q6. 阅读《Build a Large Language Model (From Scratch)》第一章

作者：Sebastian Raschka

请整理你的学习笔记。这应该包括但不限于对第一章核心概念的理解、关键术语的解析、学习过程中的思考与启发，以及尚存的疑问与反思。通过系统梳理，不仅有助于巩固自身理解，也希望为其他学习者提供有价值的参考。

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

本周学习中，主要通过自学，熟悉了python语法，通过作业题目回顾了之前在计概中学到的知识，并且通过本地部署LLM，更进一步了解了AI